

Deep Learning Solver for 2D Navier–Stokes Equations: A Physics-Informed Neural Network (PINN) Methodology

Abstract

Central Objective: I will construct a Physics-Informed Neural Network (PINN) that learns the continuous solution $u(x, y, t), v(x, y, t), p(x, y, t)$ of the 2D incompressible Navier-Stokes equations by minimizing the violation of the governing PDEs, initial conditions, and boundary conditions. I will first validate the methodology on the 1D heat equation, and then progressively scale to the 2D Navier-Stokes domain.

1 Introduction: What Are We Building

The primary objective of this project is to construct a Physics-Informed Neural Network (PINN) that approximates the solution to the 2D incompressible Navier-Stokes equations. Unlike standard data-driven deep learning models that require vast amounts of pre-computed high-fidelity data, the PINN is constrained by the underlying physical laws. The network is trained to minimize the residual of the governing PDEs, alongside the violations of Initial Conditions (ICs) and Boundary Conditions (BCs).

The proposed approach replaces the numerical discretization procedure entirely with a deep neural network. Rather than training the network on pre-existing data (data-driven), we constrain the neural network to obey the Navier–Stokes physics. The network acts as a mesh-free deep learning solver.

2 Foundation: Validation on the 1D Heat Equation

Before addressing the complexities of fluid dynamics, the fundamental PINN architecture will be validated using the 1D heat equation. Assuming thermal diffusivity $\alpha = 1$, the equation is:

$$u_t - u_{xx} = 0 \tag{1}$$

We define a neural network with parameters θ :

$$NN_{\theta}(x, t) \rightarrow \hat{u}(x, t) \tag{2}$$

Instead of solving the PDE conventionally, the network itself represents the unknown solution function. We teach the network the physics through a composite loss function.

2.1 Constructing the Loss Components

To make the neural network learn the physics, we define a loss function based on Mean Squared Error (MSE), similar to standard machine learning where $\text{MSE}(y_{\text{pred}}, y_{\text{true}})$ measures the difference between predictions and targets.

1. Physics/PDE Loss ($\mathcal{L}_{PDE}$): At any point (x, t) , the network calculates the PDE residual $r(x, t) = \frac{\partial \hat{u}}{\partial t} - \frac{\partial^2 \hat{u}}{\partial x^2}$. If the network perfectly obeys the physics, this residual should be exactly zero.

$$\mathcal{L}_{PDE} = \text{MSE}(r_{\text{pred}}, 0) \quad (3)$$

2. Initial Condition Loss ($\mathcal{L}_{IC}$): If the true initial state is $u_{\text{true}}(x, 0) = \sin(\pi x)$, we penalize the network if its prediction deviates from this truth:

$$\mathcal{L}_{IC} = \text{MSE}(\hat{u}_{\text{pred}}(x, 0), u_{\text{true}}(x, 0)) \quad (4)$$

3. Boundary Condition Loss ($\mathcal{L}_{BC}$): For boundaries where the true temperature is zero ($u_{\text{true}}(0, t) = 0$ and $u_{\text{true}}(1, t) = 0$), the loss simply penalizes any non-zero boundary predictions:

$$\mathcal{L}_{BC} = \text{MSE}(\hat{u}_{\text{boundary-pred}}, 0) \quad (5)$$

The total loss $\mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_{IC} + \mathcal{L}_{BC}$ is minimized using backpropagation, continuously updating network weights until the network accurately represents the physical system.

3 Transition to 2D Navier–Stokes

For 2D incompressible flow, the architecture expands to predict three unknown fields: x -velocity (u), y -velocity (v), and pressure (p). The neural network mapping becomes:

$$NN_{\theta}(x, y, t) \rightarrow (\hat{u}, \hat{v}, \hat{p}) \quad (6)$$

3.1 Governing Equations (Navier-Stokes)

For an incompressible Newtonian fluid, the continuity and momentum equations dictate the fluid motion:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \quad (\text{Continuity}) \quad (7)$$

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (\text{x-Momentum}) \quad (8)$$

$$\frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right) \quad (\text{y-Momentum}) \quad (9)$$

where ρ is the fluid density and ν is the kinematic viscosity.

3.2 Enforcing Fluid Physics

The physics loss is constructed using the governing equations for incompressible flow. By moving all terms to one side, we define the following residuals:

x-momentum residual:

$$r_u = u_t + uu_x + vu_y + \frac{1}{\rho} p_x - \nu(u_{xx} + u_{yy}) \quad (10)$$

y-momentum residual:

$$r_v = v_t + uv_x + vv_y + \frac{1}{\rho} p_y - \nu(v_{xx} + v_{yy}) \quad (11)$$

Continuity residual:

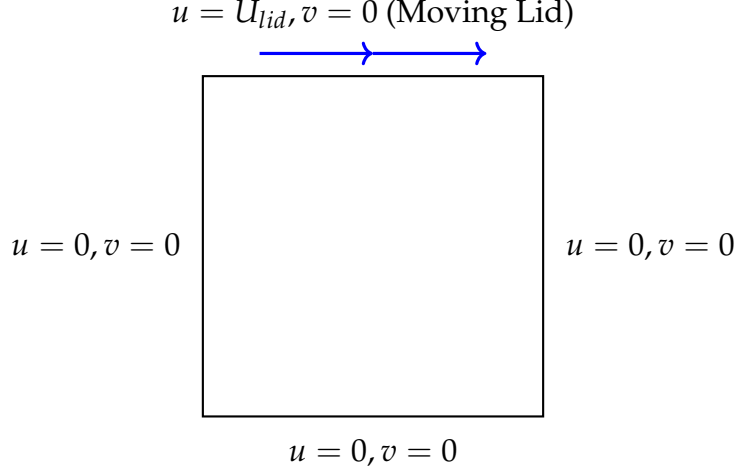
$$r_c = u_x + v_y \quad (12)$$

At random points sampled inside the fluid domain, automatic differentiation calculates all necessary spatial and temporal gradients. The physics loss forces these residuals to be zero:

$$\mathcal{L}_{NS} = \text{MSE}(r_u, 0) + \text{MSE}(r_v, 0) + \text{MSE}(r_c, 0) \quad (13)$$

3.3 Benchmark Geometry and Boundary Conditions

To rigorously validate the solver without relying on complex external data, I will utilize a standard benchmark geometry: the **Lid-Driven Cavity Flow**.



These physical constraints are codified into the boundary loss by comparing the network's boundary velocity predictions against the known true physical conditions:

$$\mathcal{L}_{BC} = \text{MSE}(u_{pred}, u_{true}) + \text{MSE}(v_{pred}, v_{true}) \quad (14)$$

4 Total Solver Architecture and Output

The total objective function combines all constraints into a single weighted loss function:

$$\mathcal{L}_{total} = \lambda_{NS}\mathcal{L}_{NS} + \lambda_{BC}\mathcal{L}_{BC} + \lambda_{IC}\mathcal{L}_{IC} \quad (15)$$

Here, the λ terms act as dynamic weights to balance the gradients between the physics loss and the boundary/initial condition losses during training.

4.1 Training Dynamics

During the initial iterations, the network outputs completely random velocity and pressure fields, resulting in massive PDE and boundary errors ($\mathcal{L}_{total} \gg 0$). Through backpropagation and an optimizer (such as Adam followed by L-BFGS for fine-tuning), the network weights are continuously updated. Over thousands of iterations, the errors diminish, and the predicted fields strictly conform to the Navier–Stokes equations and the cavity boundaries.

4.2 The Final Output: A Continuous Approximation

A critical distinction of this solver methodology is the nature of the final output. The trained model does not yield a closed-form symbolic equation, nor does it produce a discrete matrix of values locked to a specific computational grid. Instead, the trained network acts as a **continuous approximation function**:

$$NN_{\theta}(x, y, t) = [\hat{u}(x, y, t), \hat{v}(x, y, t), \hat{p}(x, y, t)] \quad (16)$$

Because the learned solution is continuous, the network can be queried at any arbitrary coordinate within the domain. For example, evaluating the network at a specific spatial and temporal point ($x = 0.2, y = 0.5, t = 1.0$) might directly yield the flow properties at that exact location:

$$NN_{\theta}(0.2, 0.5, 1.0) \rightarrow (\hat{u}, \hat{v}, \hat{p}) = (0.73, 0.12, 0.84) \quad (17)$$

By evaluating this function over thousands or millions of coordinate points, the entire flow field can be rapidly reconstructed at an arbitrarily high resolution. The neural network itself effectively becomes the PDE solver.

4.3 Architectural Overview: Neural Network Solver

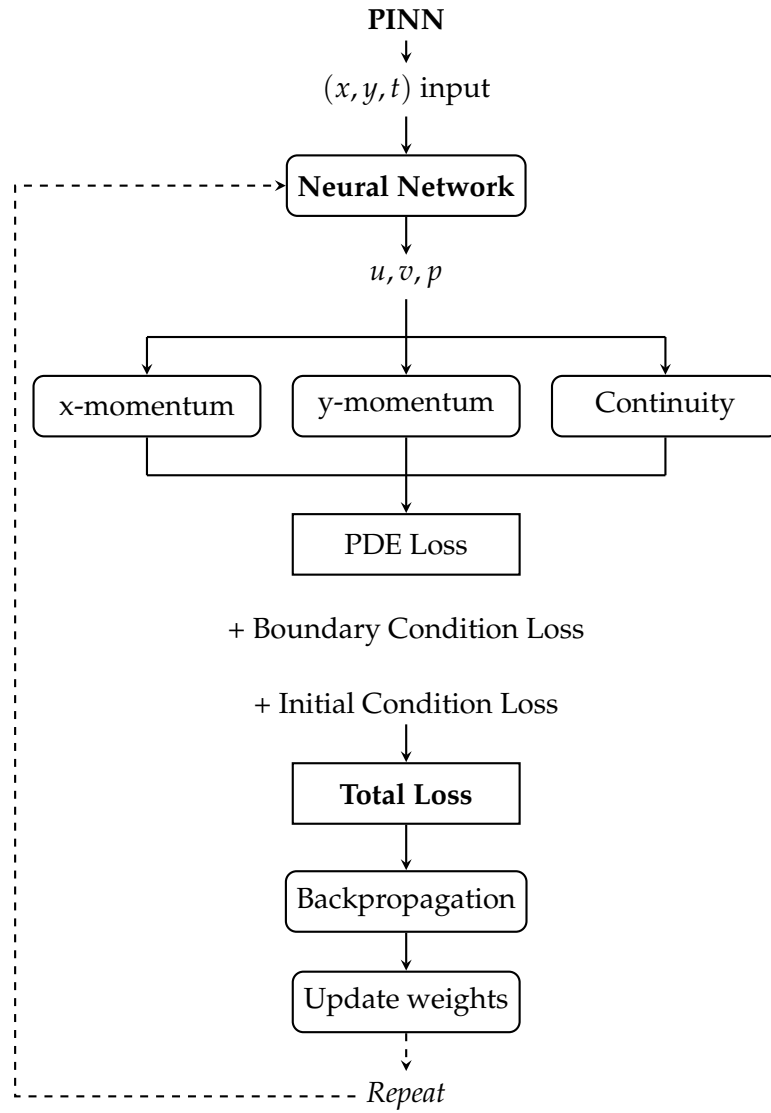


Figure 1: Formal architecture and optimization loop of the Physics-Informed Neural Network.